Personal Trainer Observability Platform - Intern Onboarding Guide

Purpose of this platform

This repository runs a complete observability and reliability platform for the ‘trainer-api‘ service.

The goal is not only to know whether a container is up. The goal is to understand whether users are getting a reliable service, how deployments affect reliability, and how engineers move from a symptom to the root cause.

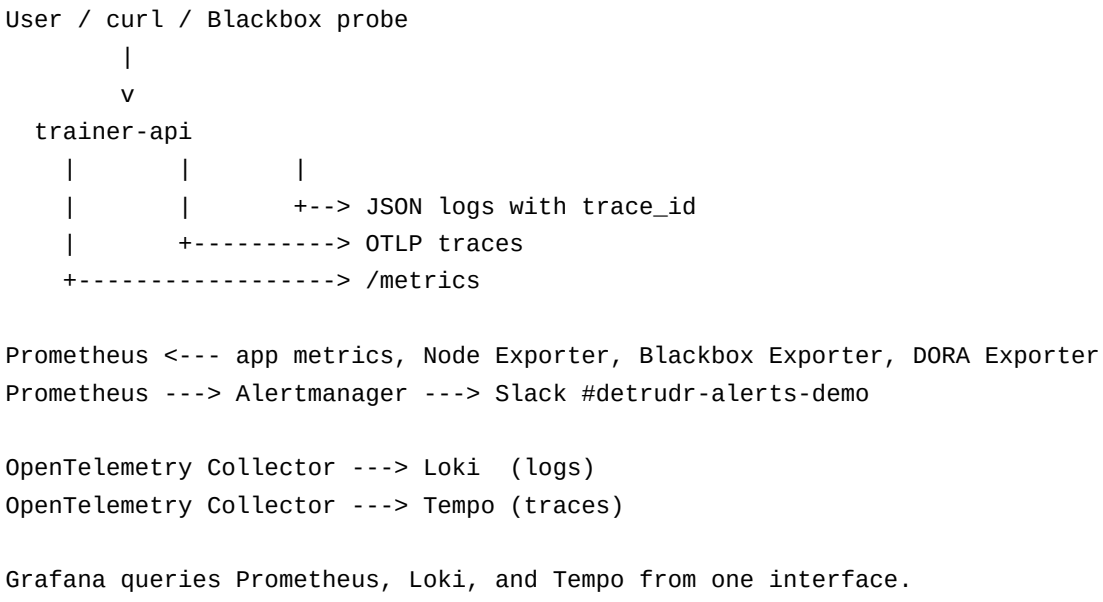
The platform uses the LGTM stack:

- Loki for logs.
- Grafana for dashboards and exploration.
- Tempo for distributed traces.
- Prometheus for metrics and alert rules.

Supporting services:

- Alertmanager routes alerts to Slack.
- Node Exporter exposes host CPU, memory, disk, network, and load metrics.
- Blackbox Exporter probes service availability, HTTP latency, and SSL expiry.
- OpenTelemetry Collector receives traces and ships logs.
- DORA Exporter exposes delivery metrics from GitHub Actions or local fallback data.
- ‘trainer-api‘ is the demo service that emits metrics, logs, and traces.

Architecture overview



Reliability drill-down flow

```
Metric spike in Grafana
        |
        v
Open matching time range in Loki logs
        |
        v
Click trace_id from the log line
        |
        v
Open the exact Tempo trace
        |
        v
Identify endpoint, span, delay, or error source
```

Service URLs

- Grafana: http://localhost:3000
- Prometheus: http://localhost:9090
- Alertmanager: http://localhost:9093
- Trainer API: http://localhost:8080
- Loki API: http://localhost:3100
- Tempo API: http://localhost:3200
- Node Exporter: http://localhost:9100
- Blackbox Exporter: http://localhost:9115
- DORA Exporter: http://localhost:9108

Important: Loki and Tempo are backend APIs, not normal browser UIs. Opening ‘http://localhost:3100‘ or ‘http://localhost:3200‘ may show ‘404 page not found‘. Use them from Grafana Explore instead.

Starting the platform

From the repository root:

```
cp terraform/terraform.tfvars.example terraform/terraform.tfvars
```

Put the Slack webhook in ‘terraform/terraform.tfvars‘:

```
slack_webhook_url = "https://hooks.slack.com/services/..."
```

Start everything:

```
make up
```

Equivalent Terraform commands:

```
terraform -chdir=terraform init
terraform -chdir=terraform apply -auto-approve
```

Useful CLI commands

Show all containers:

```
docker compose -f docker-compose.yml ps
```

Follow all logs:

```
docker compose -f docker-compose.yml logs -f --tail=100
```

Follow one service log:

```
docker compose -f docker-compose.yml logs -f --tail=100 trainer-api
docker compose -f docker-compose.yml logs -f --tail=100 otel-collector
docker compose -f docker-compose.yml logs -f --tail=100 prometheus
docker compose -f docker-compose.yml logs -f --tail=100 alertmanager
```

Restart one service:

```
docker compose -f docker-compose.yml restart trainer-api
```

Restart the full stack through Terraform:

```
terraform -chdir=terraform apply -auto-approve
```

Stop services without deleting data:

```
docker compose -f docker-compose.yml down --remove-orphans
```

Stop services and delete stored observability data:

```
docker compose -f docker-compose.yml down --volumes --remove-orphans
```

Validate Compose:

```
docker compose -f docker-compose.yml config
```

Validate application health from inside the container:

```
docker compose -f docker-compose.yml exec -T trainer-api python -c 'import urllib.request;
print(urllib.request.urlopen("http://localhost:8080/health", timeout=3).read().decode())'
```

Generate normal API traffic:

```
curl http://localhost:8080/workout
```

Generate slow requests:

```
curl "http://localhost:8080/workout?delay_ms=1200"
```

Generate errors:

```
curl "http://localhost:8080/workout?fail=true"
```

Run Game Day helpers:

```
make gameday-latency
make gameday-errors
make gameday-cpu
```

Check backend health endpoints:

```
curl http://localhost:3100/ready
curl http://localhost:3100/metrics
curl http://localhost:3200/ready
curl http://localhost:3200/metrics
curl http://localhost:9090/-/healthy
```

Grafana login

Open:

```
http://localhost:3000
```

Default credentials:

```
username: admin
password: admin
```

If credentials were changed, check ‘.env‘ or the environment values used by Docker Compose.

Grafana Explore

Grafana Explore is for ad-hoc investigation.

Use Prometheus Explore when asking metric questions:

- Is request rate increasing?
- Is error rate above normal?
- Is CPU high?
- Are Prometheus targets up?

Use Loki Explore when asking log questions:

- What did the service log during the incident?

- Are there error messages?
- Which log line has the trace ID?

Use Tempo Explore when asking trace questions:

- Which endpoint was slow?
- Which span took the longest?
- Did the request fail?

Dashboard 1: Unified Observability

This is the most important dashboard.

Use it when a service seems slow, broken, or noisy.

Panels:

- Request Rate: shows traffic by route and status code.
- Error Rate: shows the ratio of 5xx responses.
- Latency p50/p95/p99: shows request duration percentiles.
- Correlated Logs: shows Loki logs from ‘trainer-api‘.
- Recent Traces: shows Tempo traces.

How to use it:

1. Generate traffic with ‘curl http://localhost:8080/workout‘.
2. Generate latency with ‘curl "http://localhost:8080/workout?delay_ms=1200"‘.
3. Open the Unified Observability dashboard.
4. Look for a latency spike.
5. Open the logs panel for the same time range.
6. Find a log line with ‘trace_id‘.
7. Click the ‘trace_id‘ link.
8. Grafana opens the matching Tempo trace.
9. Inspect spans to identify the slow operation.

This dashboard proves the platform goes beyond simple up/down monitoring.

Dashboard 2: SLO & Error Budget

Use this dashboard to know whether the service is meeting reliability promises.

Panels:

- Availability SLI 30d: percentage of successful health probes.
- Latency SLI p95: successful request p95 latency.
- Success SLI 5m: percentage of non-5xx requests.
- Error Budget Remaining: how much allowed unreliability remains.
- Burn Rate Fast / Slow: how quickly the error budget is being consumed.
- SLO Compliance 7d / 30d: short and longer-term availability.

Key idea:

An SLI is the measurement. An SLO is the target. The error budget is the allowed failure.

Example:

```
99.5% availability over 30 days allows 0.5% failure.
30 days = 720 hours.
0.5% of 720 hours = 3.6 hours allowed failure.
```

How to use it:

1. Check whether SLI values are above their SLO targets.
2. Check whether error budget remaining is healthy.
3. If burn rate rises, open Unified Observability.
4. Use logs and traces to identify the cause.
5. Follow the ‘runbooks/slo-burn.md‘ guidance if an alert fires.

Dashboard 3: DORA Metrics

Use this dashboard to understand delivery performance.

Panels:

- Deployment Frequency: how often successful deployments happen.
- DORA Classification: Elite, High, Medium, or Low.
- Lead Time for Changes: time from commit to deployment.
- Change Failure Rate: percentage of deployments that fail or require recovery.
- Deployment Outcomes: success/failure/cancelled deployment counts.
- MTTR: average time to restore service after incidents.
- Lead Time Trend: change delivery speed over time.

How to use it:

1. Set ‘GITHUB_REPOSITORY‘, ‘GITHUB_TOKEN‘, and ‘DEPLOYMENT_WORKFLOW_NAME‘ for live GitHub Actions metrics.
2. Trigger normal deploy workflow from GitHub Actions.
3. Trigger failing workflow when testing the Game Day deployment failure scenario.
4. Watch deployment counts and change failure rate update.
5. Investigate alerts when CFR exceeds the threshold.

If GitHub credentials are not set, the DORA exporter emits fallback metrics so the dashboard still loads for demos.

Dashboard 4: Node Exporter

Use this dashboard for host-level infrastructure health.

Panels:

- CPU Usage: total host CPU.
- CPU Per Core: per-core CPU pressure.
- Memory Used/Cached/Available: memory pressure and available memory.
- Disk I/O: read and write throughput.
- Network I/O: receive and transmit throughput.
- Load Average: 1m, 5m, and 15m system load.

How to use it:

1. Open this dashboard when CPU, memory, or disk alerts fire.
2. Compare resource pressure with request latency and error rate.
3. Use ‘docker stats‘ to identify the container causing pressure.
4. If the host is unreachable, Alertmanager inhibition should reduce noisy node alerts.

Dashboard 5: Blackbox Exporter

Use this dashboard for user-facing availability.

Panels:

- Uptime / Downtime: timeline of probe success and failure.
- HTTP Response Time p50/p90/p99: external probe latency.
- SSL Expiration Days: certificate expiry countdown.
- Probe Success Rate 30d: availability from the probe perspective.

How to use it:

1. Check this dashboard when users report the service is unreachable.
2. Confirm whether the health endpoint is failing.
3. Compare probe failure time with deployment time in the DORA dashboard.
4. If probe failure lasts two minutes, ‘InstanceDown‘ should fire.

Prometheus

Prometheus is the metrics database and alert rule engine.

Open:

```
http://localhost:9090
```

Useful pages:

- ‘/targets‘: shows scrape target health.
- ‘/alerts‘: shows alert rule state.
- ‘/graph‘: lets you run PromQL manually.

Useful PromQL:

```
up
```

```
sum(rate(http_requests_total[5m])) by (route, status)
```

```
sum(rate(http_requests_total{status=~"5.."}[5m])) / sum(rate(http_requests_total[5m]))
```

```
histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le, route))
```

```
100 * (1 - avg by(instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])))
```

Loki

Loki stores logs.

Use it from Grafana Explore, not directly from the browser root URL.

Example Loki query:

```
{service_name="trainer-api"}
```

Parse JSON logs:

```
{service_name="trainer-api"} | json
```

Find logs with trace IDs:

```
{service_name="trainer-api"} | json | trace_id != ""
```

Find errors:

```
{service_name="trainer-api"} | json | levelname="ERROR"
```

Tempo

Tempo stores traces.

Use it from Grafana Explore or from clickable trace IDs in Loki logs.

Example TraceQL:

```
{ resource.service.name = "trainer-api" }
```

How to use traces:

1. Generate a request.
2. Open Grafana Explore.
3. Select Tempo.
4. Search for traces from ‘trainer-api‘.
5. Open a trace and inspect spans.
6. For latency tests, look for the ‘build_workout_plan‘ span.

Alertmanager and Slack

Alertmanager receives alerts from Prometheus and sends structured messages to Slack.

Current Slack channel:

```
#detrudr-alerts-demo
```

Open Alertmanager:

```
http://localhost:9093
```

Alert messages include:

- alert name
- severity
- service
- host or instance
- current value
- dashboard link
- runbook link
- firing or resolved status

Important config files:

- ‘observability/prometheus/rules/infrastructure.yml‘
- ‘observability/prometheus/rules/slo-burn.yml‘
- ‘observability/prometheus/rules/dora.yml‘
- ‘observability/alertmanager/alertmanager.yml‘
- ‘observability/alertmanager/templates/slack.tmpl‘

Runbooks

Runbooks live in ‘runbooks/‘.

Use runbooks when alerts fire.

Current runbooks:

- ‘runbooks/cpu.md‘
- ‘runbooks/memory.md‘
- ‘runbooks/disk.md‘
- ‘runbooks/instance-down.md‘
- ‘runbooks/slo-burn.md‘
- ‘runbooks/change-failure-rate.md‘
- ‘runbooks/mttr.md‘

Every runbook explains:

- what the alert means
- likely causes
- first investigation steps
- resolution steps
- rollback guidance
- escalation path

Common troubleshooting

If Terraform says "No changes" but containers are not running:

```
terraform -chdir=terraform apply -auto-approve
```

The Terraform resource is configured to re-run Compose on each apply.

If a container keeps restarting:

```
docker compose -f docker-compose.yml ps
docker compose -f docker-compose.yml logs --tail=120 SERVICE_NAME
```

If Loki or Tempo returns 404 in the browser:

That is normal at the root path. Use Grafana Explore.

If Grafana dashboards are missing:

```
docker compose -f docker-compose.yml restart grafana
docker compose -f docker-compose.yml logs --tail=100 grafana
```

If Slack alerts do not arrive:

1. Confirm ‘terraform/terraform.tfvars‘ contains the correct webhook.
2. Confirm Alertmanager is running.
3. Open ‘http://localhost:9093‘.
4. Check Alertmanager logs.
5. Confirm the webhook app can post to ‘#detrudr-alerts-demo‘.

If Prometheus targets are down:

1. Open ‘http://localhost:9090/targets‘.
2. Find the failed job.
3. Check the matching container with ‘docker compose ps‘.
4. Read that container’s logs.

What a junior intern should practice first

1. Start the stack with ‘make up‘.
2. Open Grafana.
3. Generate normal traffic with ‘/workout‘.
4. Generate latency with ‘delay_ms=1200‘.
5. Watch latency rise in Unified Observability.
6. Open Loki logs for the same time window.
7. Click a ‘trace_id‘ into Tempo.

8. Open Prometheus targets and alerts pages.
9. Trigger an error with ‘fail=true‘.
10. Read the SLO burn runbook and explain what would happen if the error continued.

Mental model

Prometheus tells you what changed.

Loki tells you what the service said.

Tempo tells you where time was spent.

Grafana brings the three views together.

Alertmanager turns reliability problems into actionable Slack notifications.

Runbooks tell the responder what to do next.